

A Beginner's Guide To Exploratory Data Analysis (EDA) With Real Examples

medium.com/@tararasti/a-beginners-guide-to-exploratory-data-analysis-eda-with-real-examples-535424f3dafe

Tara Rasti

May 26, 2025



Photo by [Isaac Smith](#) on [Unsplash](#)

In this article I am going to explain a step-by-step Exploratory Data Analysis (EDA) in python using the **California Housing Dataset** from `sklearn.datasets` and **Titanic Dataset** from `seaborn`.

EDA pioneered by [John Tukey](#), set a foundation for the field of data science. The key idea of EDA is that the first and most important step in any project based on data is to *look at the data*. By summarising and visualising the data, you can gain valuable intuition and understanding of the project. EDA should be a cornerstone of any data science project.

🔍 What is EDA?

EDA is the process of examining and understanding a dataset before applying any modelling or machine learning. It's like getting to know your data — its structure, patterns, anomalies, and relationships — so you can make informed decisions in later stages.

There are 4 steps in EDA:

1. Loading the dataset
2. Summary statistics
3. Missing values check
4. Visualisation (distributions, correlation heatmap, scatterplots)

In sections 1 to 5 I will explain EDA for single variables. In section 6 we will go through EDA for more than one variable.

1. Loading The Dataset

California Housing Dataset was derived from the 1990 U.S. census, using one row per census block group. A block group is the smallest geographical unit for which the U.S. Census Bureau publishes sample data. This dataset does not contain true categorical variables in its original form. Therefore for analysing categorical variables I will use another dataset named Titanic dataset in section 5. But first let's load the California Housing dataset.

```
sklearn.datasets.fetch_california_housing pandas pdcalifornia =  
fetch_california_housing(as_frame=True)df = pd.DataFrame(california.data,  
columns=california.feature_names)df['target'] = california.target(df.head())(california.DESCR)
```

2. Summary Statistics

2.1. Basic Info

To gain a quick understanding of the dataset's structure and contents, we start by exploring its shape, data types, and a preview of the first few rows.

```
(df.shape)(df.dtypes)(df.head())(df.describe())
```

The function `df.describe()` provides summary statistics for the numerical columns of a DataFrame `df`.

2.2. Estimate of Location

Variables with measured or count data might have thousands of distinct values. A basic step in exploring the data is getting a "typical value" for each feature (variable): an estimate of where each data is located (i.e., its central tendency).

```
scipy.stats.trim_mean()( * ) col df.columns: mean_val = df[col].mean()  
trimmed_mean_val = trim_mean(df[col], proportiontocut=) median_val = df[col].median()  
( )
```

- : The sum of all values divided by the number of values.
- : The average of all values after dropping a fixed number of extreme values.

- : The value such that one-half of the data lies above and below.

The basic metric for location is the mean, but it can be sensitive to extreme values (outliers). Other metrics (median, trimmed mean) are less sensitive to outliers and unusual distributions and hence are more robust.

2.3. Estimate of Variability 📊

Location is just one dimension in summarising a feature. A second dimension, variability, also referred to as dispersion, measures whether the data values are tightly clustered or spread out.

```

import numpy as np
import pandas as pd
import scipy.stats as stats

df = pd.DataFrame({'col': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]})

iqr_val = stats.iqr(df[col])
mad_val = stats.median_abs_deviation(df[col])
std_dev = df[col].std()

```

- : The sum of squared deviations from the mean divided by $n-1$ where n is the number of data values.
- : The square root of the variance.
- : The median of the absolute values of the deviations from the median.

The standard deviation is much easier to interpret than the variance since it is on the same scale as the original data.

Neither the variance, nor the standard deviation is robust to outliers and extreme values. A robust estimate of variability is the MAD. Like the median, the MAD is not influenced by extreme values.

3. Missing values check 🕒

For checking the NA values in a dataset you can use the `is.null()` function from Pandas.

```
df.isnull().sum()
```

4. Visualisation 📈

4.1. Boxplot 📦

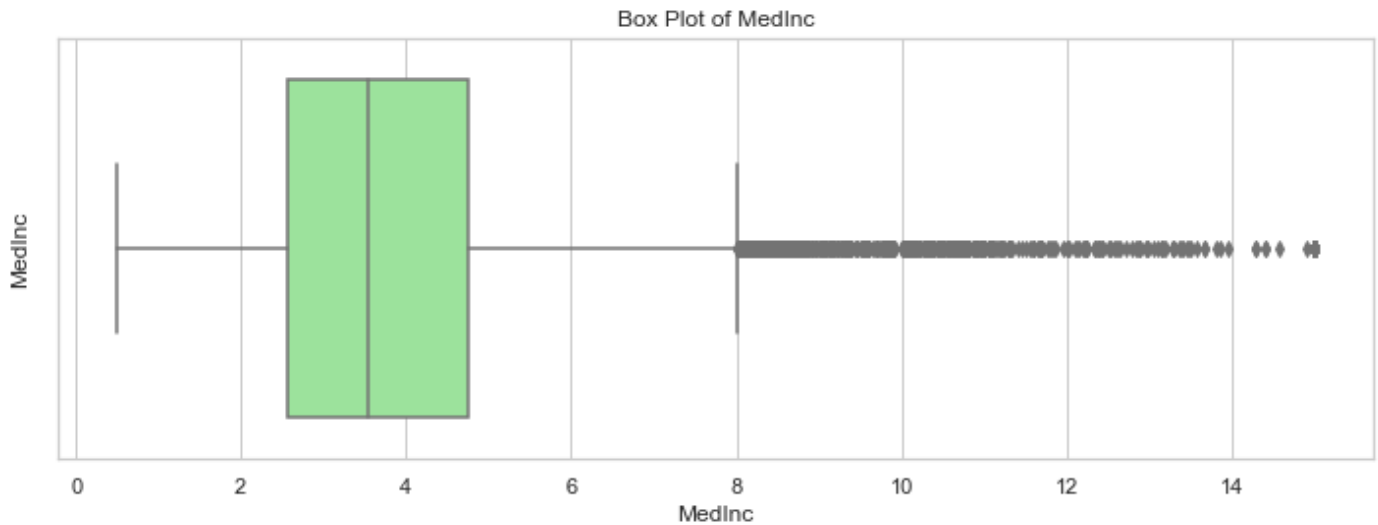
The percentile is the value such that P percent of the values take on this value or less and $(100-P)$ percent take on this value of more. A boxplot — with the top and bottom of the box at the 75th and 25th percentiles, respectively — also gives a quick sense of the distribution of the data; it is often used in side-by-side displays to compare distributions.

```

import matplotlib.pyplot as plt
import seaborn as sns

sns.boxplot(x=df[col], color='red')
plt.title('Boxplot of col')
plt.ylabel(col)
plt.tight_layout()
plt.show()

```

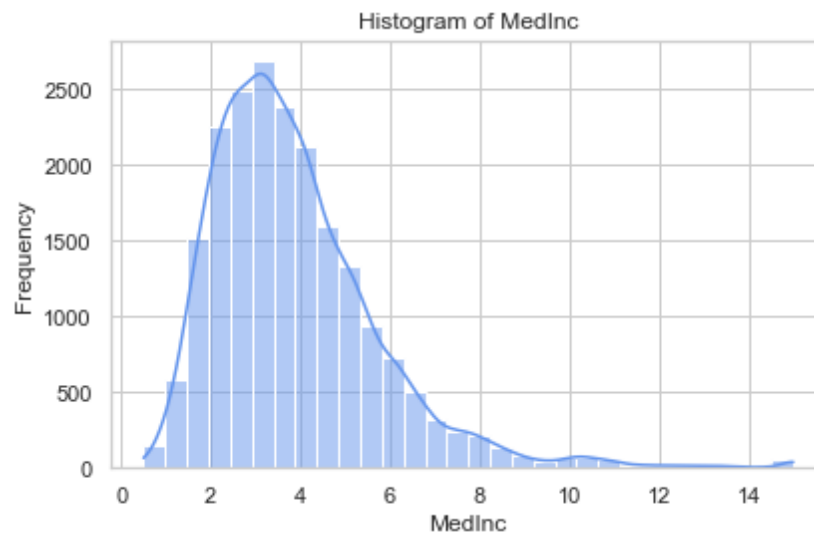


The Boxplot for MedInc (median income in block group)

4.2. Histogram

A frequency histogram plots frequency counts on the y-axis and variable values on the x-axis; it gives a sense of the distribution of the data as a glance.

```
sns.(style=) col df.columns: plt.figure(figsize=(, )) sns.histplot(df[col], bins=,
kde=, color=) plt.title() plt.xlabel(col) plt.ylabel() plt.tight_layout()
plt.show()
```



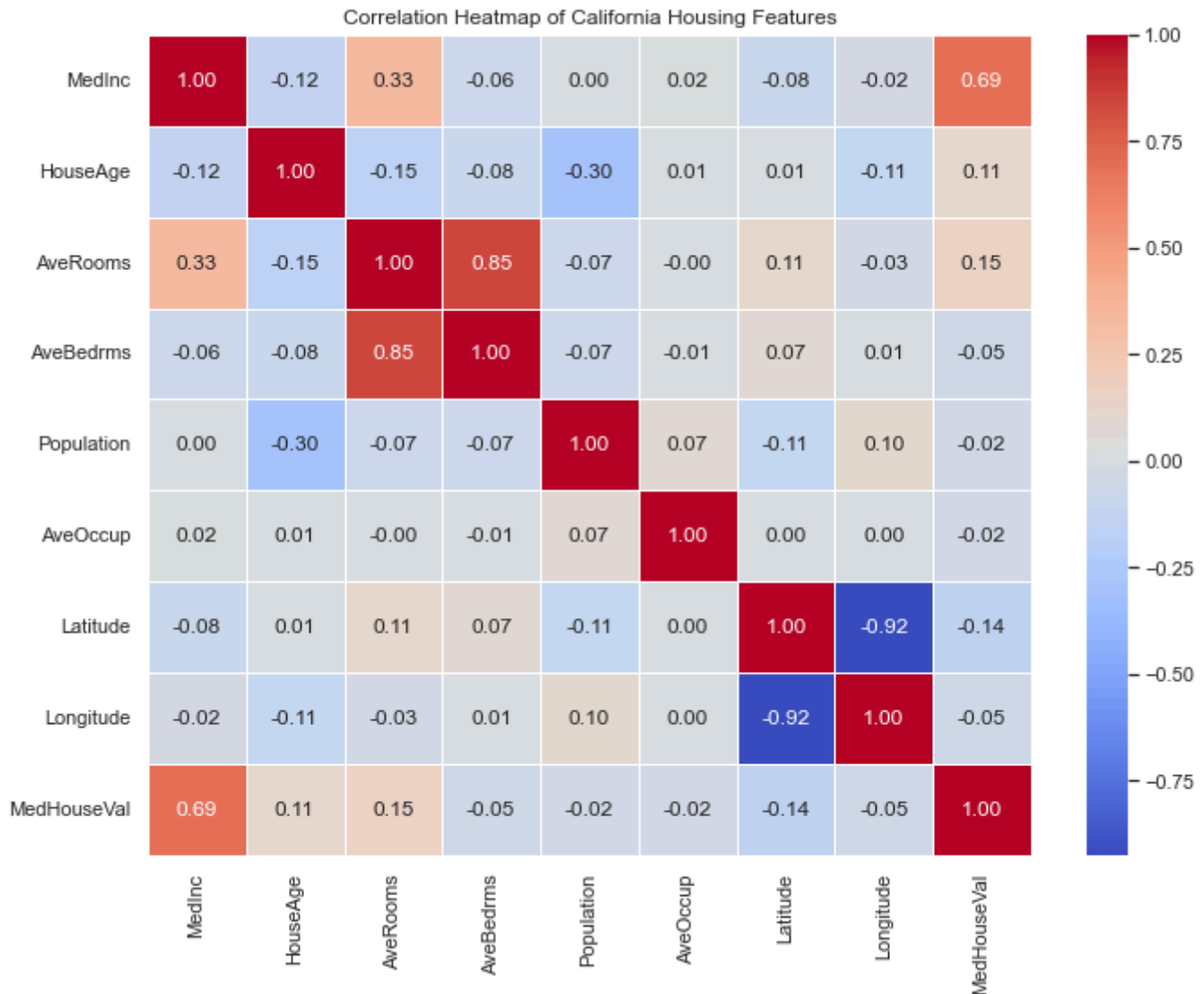
The Histogram for MedInc

4.3. Correlation

EDA in many projects involves examining correlation among predictors, and between predictors and a target variable. Correlation coefficient is a metric that measures the extent to which numeric variables are associated with one another (ranges from -1 to +1). A correlation coefficient of zero indicates no correlation.

A table where the variables are shown on both rows and columns, and the cell values are the correlations between the variables.

```
corr_matrix = df.corr()plt.figure(figsize=(, ))sns.heatmap(corr_matrix, annot=, cmap=,
fmt=, linewidths=)plt.title()plt.tight_layout()plt.show()
```



Correlation Heat Map for California Housing Dataset

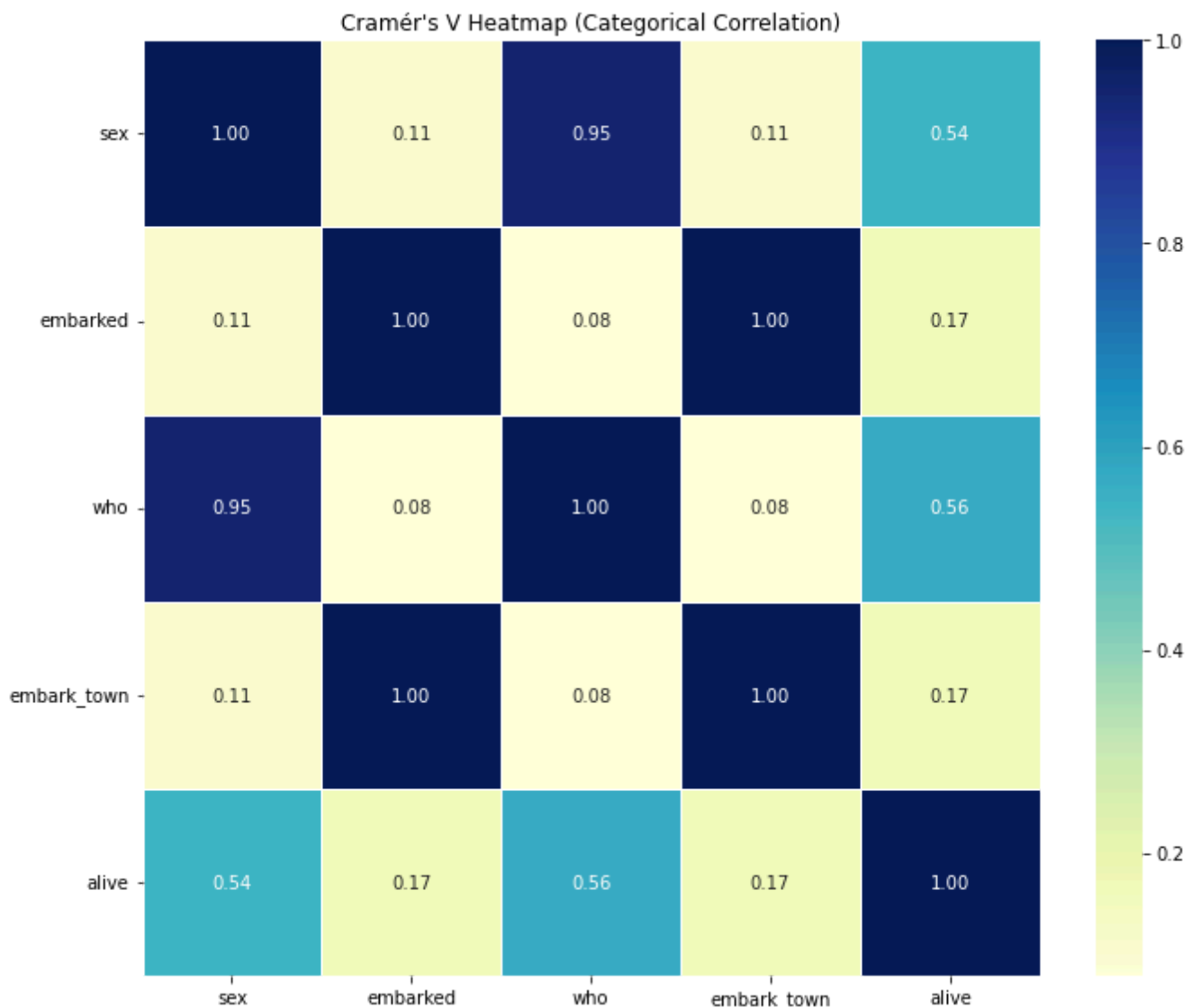
4.4. Correlation for Categorical Variables abcd

Seaborn's heatmap is designed for numerical correlations. However, you can analyse associations between categorical variables by using statistical measures such as Cramér's V or Theil's U instead of standard correlation coefficients. Here's how to compute Cramér's V for all pairs of categorical variables and plot the results in a heatmap.

```

pandas pd seaborn sns matplotlib.pyplot plt numpy np scipy.stats
chi2_contingency(titanic = sns.load_dataset()) cat_cols =
titanic.select_dtypes(include=).columns.tolist() (): chi2 =
chi2_contingency(confusion_matrix)[0] n = confusion_matrix.shape[0] phi2 = chi2 / n
r, k = confusion_matrix.shape phi2corr = (phi2 - ((k - 1)*(r - 1)) / (n - 1)) rcorr = r
- ((r - 1)**2) / (n - 1) kcorr = k - ((k - 1)**2) / (n - 1) np.sqrt(phi2corr / ((kcorr - 1),
(rcorr - 1))) cramers_results = pd.DataFrame(index=cat_cols, columns=cat_cols) col1
cat_cols: col2 cat_cols: confusion_mat = pd.crosstab(titanic[col1],
titanic[col2]) cramers_results.loc[col1, col2] =
cramers_v(confusion_mat) cramers_results = cramers_results.astype(float) plt.figure(figsize=(
)) sns.heatmap(cramers_results, annot=True, fmt='.2f', cmap='magma',
linewidths=1) plt.title('Cramér's V Heatmap (Categorical Correlation)') plt.xticks(rotation=45) plt.yticks(rotation=45) plt.tight_layout() plt.show()

```

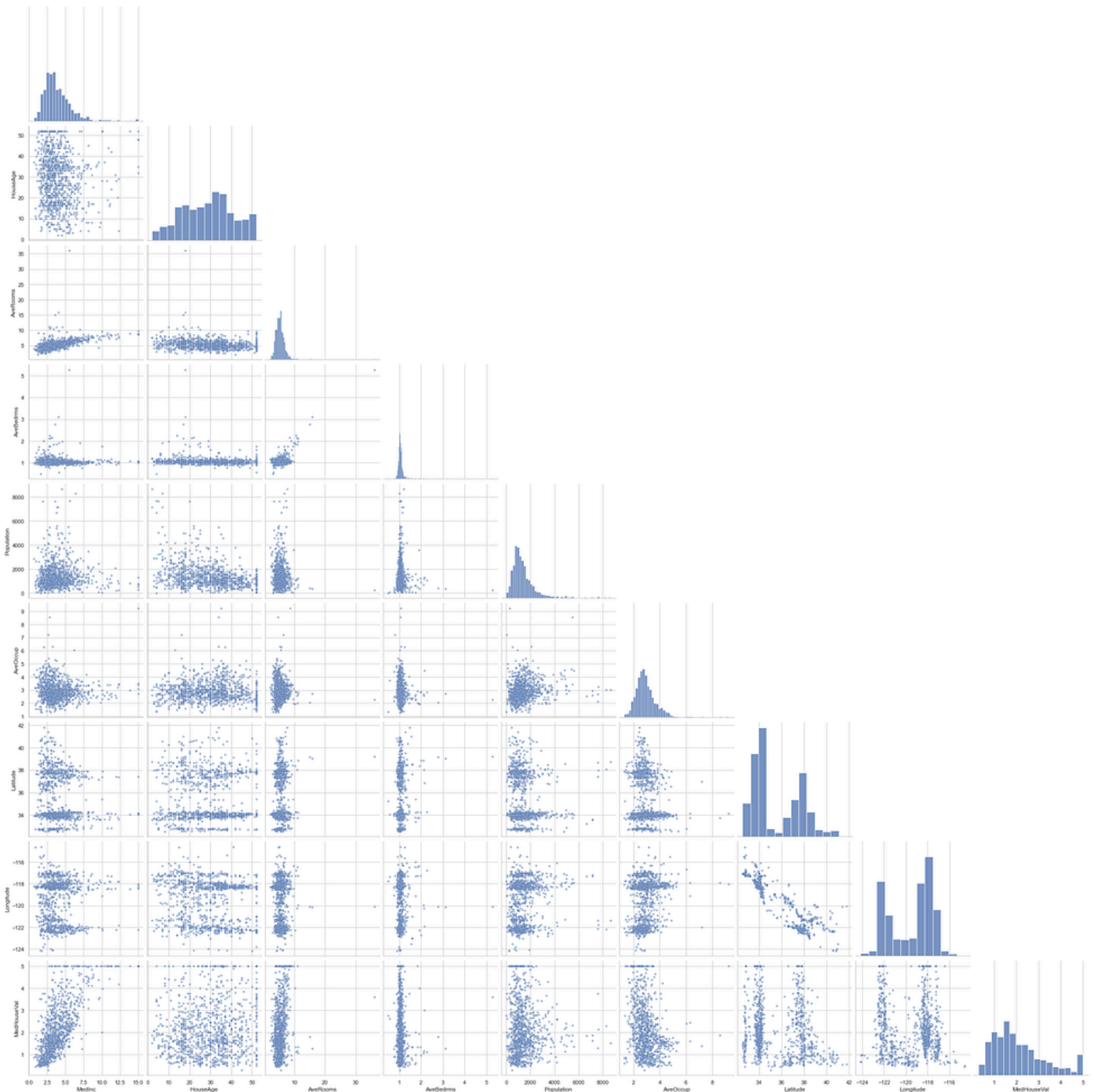


4.4. Scatter Plot ●

Scatter plot is a kind of plot in which the x-axis is the value of one variable, and the y-axis the value of another.


```
df_sample = df.sample(, random_state=)sns.(style=)plot = sns.pairplot( df_sample,
corner=, height=, plot_kws={: , : } )plot.fig.suptitle(, y=)plt.show()
```

Large Scatter Plot Matrix of California Housing Dataset



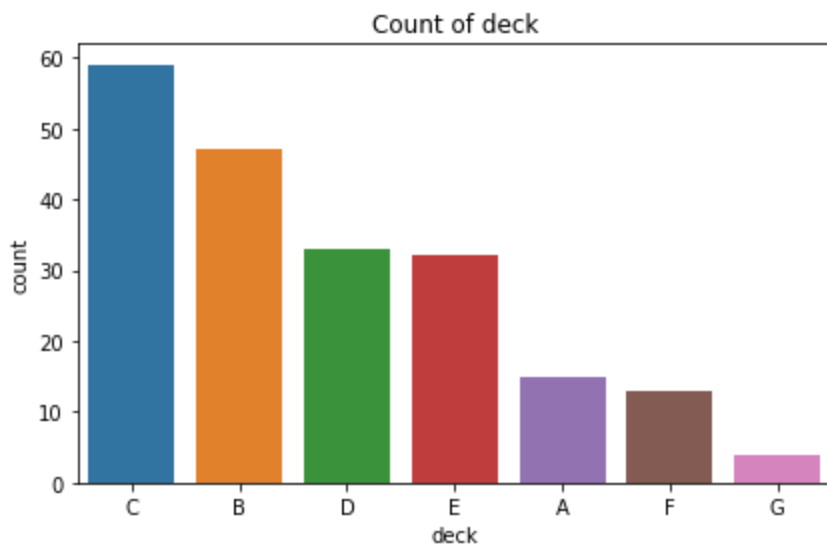
5. Exploring Binary and Categorical Data 🏠

For categorical data , simple proportions or percentages tell the story of the data.

```
titanic = sns.load_dataset()(titanic.head())
```

5.1. Barchart

```
matplotlib.pyplot plt seaborn sns col categorical_cols: plt.figure(figsize=(,))
sns.countplot(data=df, x=col, order=df[col].value_counts().index) plt.title()
plt.xticks(rotation=) plt.tight_layout() plt.show()
```

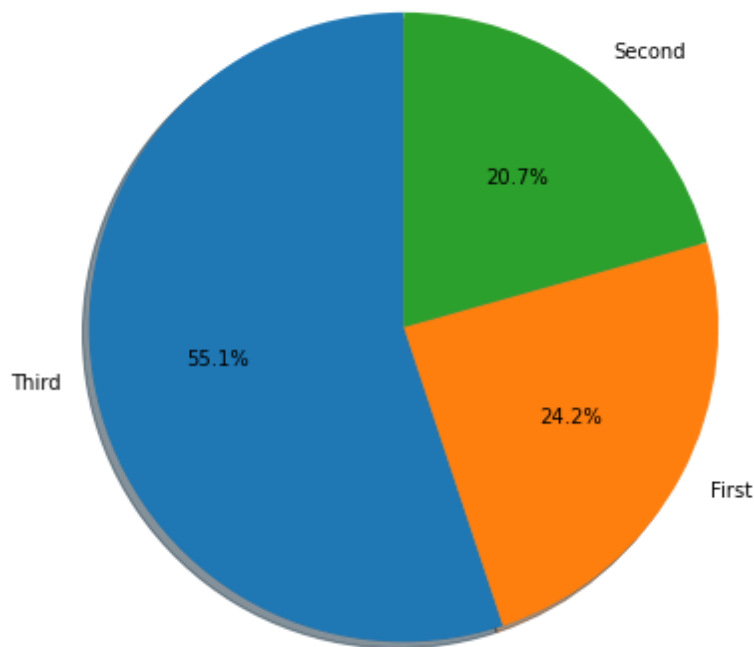


Barchart for Deck

5.2. Pie Chart

```
col categorical_cols: plt.figure(figsize=(,))
df[col].value_counts(dropna=).plot.pie( autopct=, startangle=,
shadow=, figsize=(, ), label= ) plt.title() plt.ylabel()
plt.tight_layout() plt.show()
```


Distribution of class



Pie chart of Class

- : The most commonly occurring category or value in a dataset.
- : The frequency or proportion of each category plotted as bars.
- : The frequency or proportion for each category plotted as wedges in a pie.

Note that a bar chart resembles a histogram; in a bar chart the x-axis represents different categories of a factor variable, while in a histogram the x-axis represents values of a single variable on a numeric scale.

6. Exploring Two or More Variables

Familiar estimators like mean and variance look at variables one at a time (univariate analysis). Correlation analysis is an important method that compares two variables (bivariate analysis). In this section we look at additional estimates and plots, and at more than two variables (multivariate analysis).

6.1. Contingency Table

A useful way to summarise two categorical variables is a contingency table — a table of counts by category. A contingency table is tally of counts between two or more categorical variables. Contingency tables can look only at counts, or they can also include column and total percentages.

```

itertools combinations scipy.stats statstitanic = sns.load_dataset()categorical_cols =
titanic.select_dtypes(include=[, , ]).columns.tolist() (): chi2 =
stats.chi2_contingency(confusion_matrix, correction=)[] n = confusion_matrix.().()
phi2 = chi2 / n r, k = confusion_matrix.shape np.sqrt(phi2 / (k - , r -
))cramers_v_matrix = pd.DataFrame(index=categorical_cols, columns=categorical_cols) var1,
var2 combinations(categorical_cols, ): table = pd.crosstab(titanic[var1],
titanic[var2]) v = cramers_v(table) cramers_v_matrix.loc[var1, var2] = (v, )
cramers_v_matrix.loc[var2, var1] = (v, )np.fill_diagonal(cramers_v_matrix.values, )()
(cramers_v_matrix)

```

6.2. Boxplot for Categorical and Numeric Data 🎨

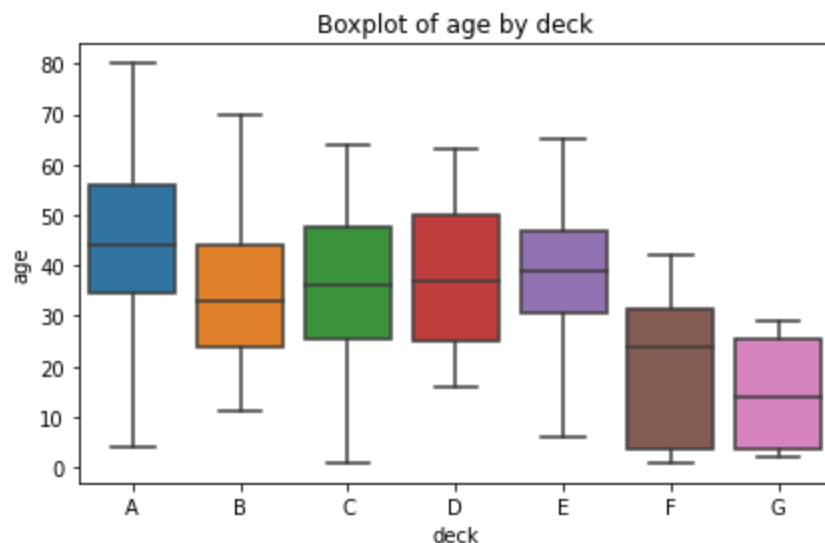
Boxplots are simple way to visually compare the distributions of a numeric variable grouped according to a categorical variable.

Here I wrote a code for evaluating the numeric variable Age based on all categorical variables.

```

categorical_cols = titanic.select_dtypes(include=[, , ]).columnsnumerical_var = col
categorical_cols: plt.figure(figsize=(, )) sns.boxplot(x=col, y=numerical_var,
data=titanic) plt.title() plt.xticks(rotation=) plt.tight_layout() plt.show()

```



Boxplot of Age based on Deck variable

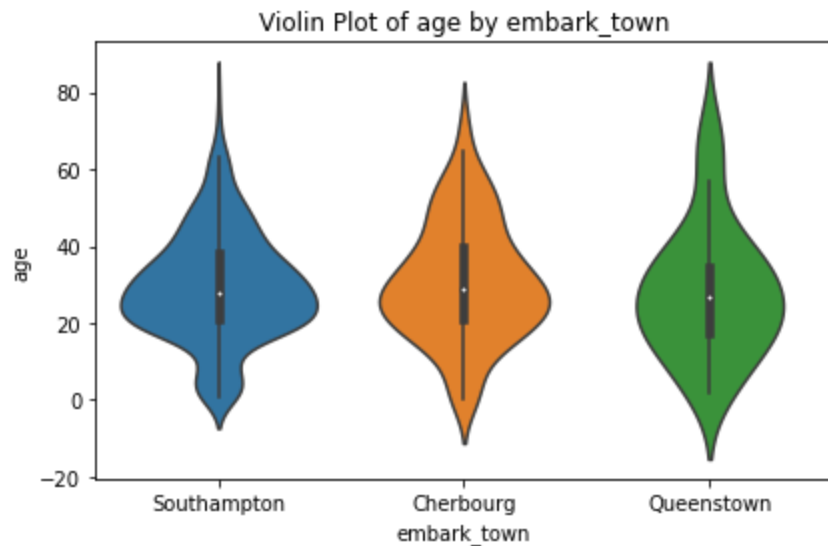
6.3. Violin Plot 🎻

A violin plot is an enhancement to the boxplot and plots the density estimate with the density on the y-axis. The density is mirrored and flipped over, and the resulting shape is filled in, creating an image resembling a violin.

```

categorical_cols = titanic.select_dtypes(include=[, , ]).columnsnumerical_var = col
categorical_cols: plt.figure(figsize=(, )) sns.violinplot(x=col, y=numerical_var,
data=titanic, inner=) plt.title() plt.xticks(rotation=) plt.tight_layout()
plt.show()

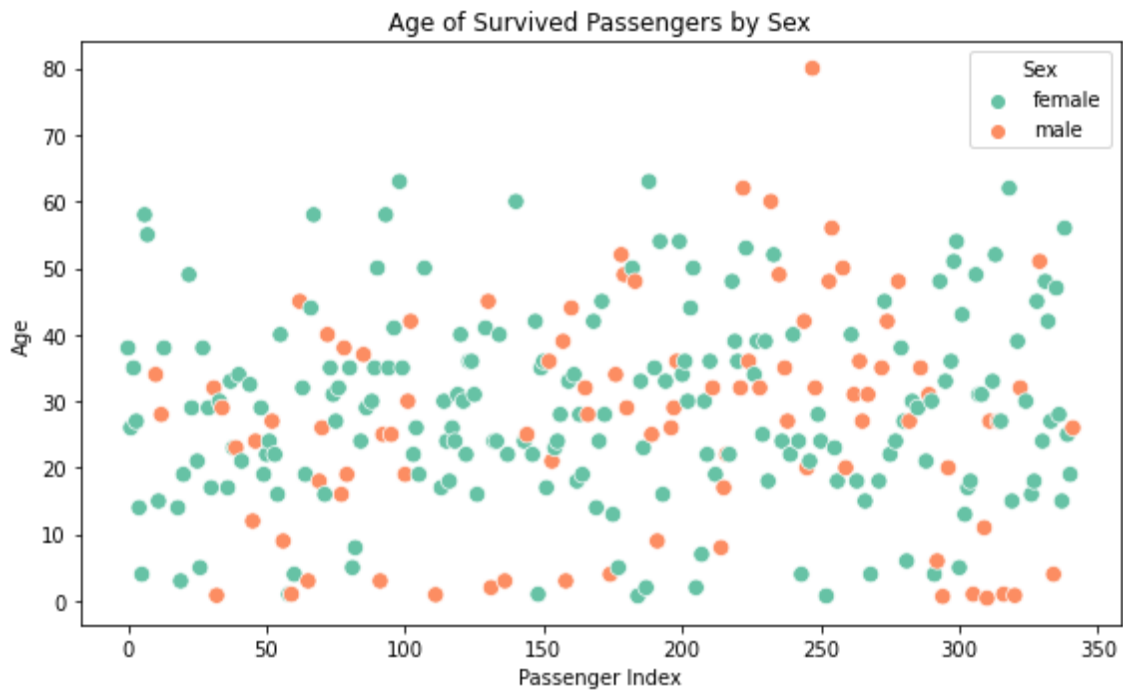
```



6.4. Visualising Multiple Variables

The types of charts used to compare two variables — scatterplots, and boxplots — are readily extended to more variables through the notion of conditioning. Here I used this code to draw a scatterplot for the age of survived passengers.

```
survivors = titanic[titanic[] == ].reset_index(drop=)
survivors[] = survivors.index
plt.figure(figsize=(, ))
sns.scatterplot(x=, y=, hue=, data=survivors, palette=,
s=)
plt.title()
plt.xlabel()
plt.ylabel()
plt.legend(title=)
plt.tight_layout()
plt.show()
```



Scatter Plot of Age of Survived Passengers by Sex

✓ Final Thoughts

Exploratory Data Analysis helps you build intuition and trust in your data. It reveals trends, errors, and relationships that are crucial before applying models. Think of EDA as **having a conversation with your data** — one where you listen first.



EDA is an essential first step in any data project.

Summary stats, missing values, and visualization help you understand data patterns.

Tools like Pandas, Seaborn, and Matplotlib are invaluable in EDA.

Did I miss your favorite EDA technique? Let me know in the comments!

If you found this helpful, **clap** 👏 and **follow** for more data science guides.

To find out more about the codes click on my [GitHub link](#).

“I’m a data science enthusiast writing about Python, machine learning, and real-world projects. Reach me on [LinkedIn](#) or follow for future articles.”